PHANTOM: Causal Attribution of Runtime SBOM Drift via eBPF Behavioral Contracts in Kubernetes

Harsh Ranjan
Vellore Institute of Technology
Email: harsh.ranjan2023@vitstudent.ac.in

Jai Goel
Vellore Institute of Technology
Email: jai.goel2023@vitstudent.ac.in

Dr. Thanuja R
Guide, Vellore Institute of Technology
Email: thanuja.r@vit.ac.in

Abstract, Cloud-native applications are counting on third-party components whose runtime behavior can change after de-
ployment. Monitors and software bills of materials (SBOMs) encapsulate distinct indication; linking events to components and causal responsibility remains difficult. My point is, PHANTOM implements SBOM verification interfaces, eBPF event collection, variable-
order Markov behavioral contracts, a time-decayed Behavioral Dependency Graph (BDG), and a DoWhy-based causal-
estimation pathway and an XGBoost PCEPS scorer, in actual application. Its design combines image- and PURL-scoped contract generation with a temporal causal projection that returns not_identifiable when stipulated identification checks fail; this is an integration design, not a claim that those mechanisms are new or that component- level causal attribution has been validated. In an evaluation of six completed attack runs across the two tested supply-
chain attack families, drift events with mean latency 15.74 seconds, were recorded median 5.52 seconds by PHANTOM, and linearly interpolated 95th percentile 54.72 seconds. Three SolarWinds-
style injections failed before an attack phase. The saved event records lack settled PURL/SBOM bindings, and the dataset comprises no benign controls; thus attribution accuracy, false-
positive rate, adjusted prediction, and runtime overhead remain unestablished; these initial findings document event presence and latency only. PHANTOM contributes an implementable, uncertainty-aware research pathway whose component mapping and causal value need controlled eval-
uation.

Index Terms, eBPF, software supply chain, causal inference, Kubernetes, runtime security, SBOM

I. Introduction

Cloud-native applications are gathering services from third-
party packages, container images, build tools, and de-
ployment configuration; that composition shortens development cycles; it also is creating a few points where the delivered software can differ from what an operator anticipated. A reliance may be swapped, a build artifact may be altered. And yet, a legitimate update may change runtime behavior. Malicious functionality was introduced into se-
lected liblzma release artifacts by modified build scripts. The snag is shown by the XZ Utils incident. Neither view alone answers whether an noticeable process, file access, or network connection came from a particular package. An SBOM records component inventory and relationships; runtime monitoring is seeing what carries out. But still, whether a package change caused the behavior. And more, the resulting behavior impacted an OpenSSH authentication path. The compro-
mise became public after Andres Freund assessed anomalous CPU use and related symptoms in SSH testing [1], [2]. A runtime event such as an unexpected execution or network connection could be helpful proof in a akin incident. But that event alone wouldn't spot the responsible package. A process may load code from several packages, inherit configuration from its workload, or perform unusual behavior after a legitimate rollout change. Attribution therefore rests on provenance mapping and clear assumptions about the causes of gathered behavior. Like, when a runtime monitor reports an unexpected execution, its event record may spot the process and operation. PHANTOM studies an architecture for joining these evidence types: contracts keyed by image digest and PURL, event relationships in a Behavioral Dependency Graph (BDG), and a causal-
estimation pathway that abstains when configured identification checks fail. The record doesn't by itself establish whether code associated with ‘pkg:pypi/lz4@4.3.2‘ caused the execution. Well the same, whether a configuration change in the same container is explaining it. The available attack records don't include tackled PURL/SBOM bindings. But still, the intended component-level link is implemented as a system pathway but isn't showed by these experiments; this paper implements and evaluation clues, with evidence scopes stated separately:

- Implemented and gathered at the event level: the store is comprising a variable-order Markov contract- generation pipeline, BIC order selection, and KL- divergence scoring. In six completed attack runs across two families, the saved

entries include drift events within the recorded detection windows; their median injection-to-event latency is 5.52 s. The records don't establish that these events were bound to the responsible SBOM component. • Implemented and tested structurally: the repository includes a typed, time-decayed BDG and a temporal DAG projection with a fixed semantic tier ordering. The local test run completed with 454 passed and one skipped test across the core library and five services; these unit tests exercise code paths, covering cycle handling and abstention cases; they don't prove causal correctness or component attribution in deployment.

• Implemented, awaiting experimental validation: a DoWhy adapter and a 16-feature PCEPS training, calibration, and scoring pathway are present. The adapter returns not_identifiable for stipulated failures rather than using a correlation fallback. The saved run artifacts encompass no component-level oracle labels, trained scoring model, or held-out predictive findings.

A. Design Distinctions and Research Claims

PHANTOM's research question concerns the interfaces among event collection, package identity, behavioral deviation, and causal analysis; the use of eBPF, Markov models, graphs, DoWhy, or XGBoost alone isn't claimed as novel. The implementation makes three testable design choices. First, contract-generation requests carry an image digest and PURL, allowing component-scoped model entries when event-to-component identity is available. Second, the causal projection orients selected within-window edges using an explicit semantic tier hierarchy, while preserving the coring BDG and withholding queries that remain cyclic. Third, identification failures generate the first-class status not_identifiable, rather than an association score presented as a causal effect. These are implementation properties and candidate research contributions, not a verified claim that PHANTOM is the first framework to make them or that the tier assumptions recover true causal structure. The saved experiment records lack the package bindings and autonomous labels needed to test whether these choices raise attribution. The check is intentionally narrower than a detector benchmark. Six completed attacks produced event-positive records; three SolarWinds-style injections failed before an attack phase. No harmless control windows were saved, so false-positive rate and precision can't be calculated. The baseline log documents a dry run rather than comparable Falco, Trivy, or Isolation Forest results. Mean latency was 15.74 s, median 5.52 s, and linearly interpolated P95 54.72 s; the first XZ run took 70.332 s, and the available logs don't explain the delay. Reporting these limits makes clear which notes the present artifacts support and which research questions remain open; it doesn't turn the six-run dataset into the proof of broad detection performance. Section II reviews supply-chain attacks, runtime telemetry, SBOMs, and causal analysis. Section III sketches the threat model, contract pipeline, BDG, causal projection, and PCEPS. Section IV recaps the services and analyst interface. Section V reports the saved evaluation indication and its shortcomings, including the controls and baselines still needed. Section VI concludes with the implementation contribution and the next validation steps.

II. Background and Related Work

A. Software Supply Chain Attacks

Supply-chain compromises exploit trust relationships between producers, build systems, and consumers. SolarWinds uncovered SUNBURST in December 2020 after malicious code had been inserted into the Orion build pipeline and distributed inside digitally signed updates. The backdoor remained inactive before beginning selective network communications, and its delivery through a trusted vendor update complicated detection based on package reputation alone [3], [4]. Log4Shell, shared in December 2021 as CVE-2021-44228, was mixed: attacker-controlled strings could trigger JNDI lookups in vulnerable Log4j configurations, potentially directing to remote code execution. The library was extensively embedded transitively, so asset owners often didn't know which services loaded it [5], [6]. The XZ Utils compromise became public on March 29, 2024, when a developer found unusual CPU behavior during SSH testing. Modified build scripts enabled a backdoor in chosen release artifacts, despite source review and package provenance appearing ordinary to many downstream consumers [1], [2]. These incidents differed in mechanism, but each challenged a different monitoring assumption: SUNBURST used delayed, selective execution; Log4Shell was switched on by input and reliance reachability; XZ altered build outputs and focused a sensitive library interaction. At the kernel boundary, each could produce observable evidence such as unexpected process execution, file access, or outbound connections, but no single event signature would uniquely establish the responsible package. Kernel evidence can narrow the explanation; it can't by itself prove software provenance.

B. eBPF-Based Runtime Security

Falco uses eBPF in cloud-native systems to monitor Linux and report suspicious activity [7]. It lets users create rules through a policy rule engine, which is helpful for recognizing known malicious patterns and policy violations. Its coverage, though, depends on the rules available: behavior outside the configured rules may go undetected. Tetragon also uses eBPF to monitor operating-system activity, including process execution and network use, for security observability and policy enforcement [8]. It awarely controls, while linking its events to a specific SBOM component requires extra identity mapping and examination. Tracee offers Linux runtime security and forensic capabilities using eBPF, including event collection and policy-based detection [9]. These tools can provide

valuable evidence and enforcement, while package-level attribution depends on the rolled out configuration and any identity-
mapping integrations. Collecting events or enforcing a policy doesn't by itself estimate whether a particular component change caused an gathered departure.

C. Software Bill of Materials and Runtime Drift

CycloneDX defines a machine-readable bill of materials for software components, reliances, and related supply-chain metadata [10]. A practical baseline for SBOM fields and exchange [11] was also established by the NTIA minimum-elements work. Such formats make component invento-
ries more good, but a clear remains a statement about declared or gathered contents at a point in the build or scan process. It isn't a continuous record of behaviour after rollout; in turn, runtime evidence and sbom enforcement have been connected by prior work. Bomfather uses kernel-level eBPF observa-
tion during the builds to spot reliances and build build artifacts; its scope is build-time dependency dis-
covery rather than the post-deployment drift and causal-estimation process scrutinized here [29]. Crawford emphet al. describe enforcing SBOM constraints through Linux eBPF and IMA [30], so these efforts make clear that runtime/SBOM integration isn't, by itself, PHANTOM's novelty. The store instead builds a combination of image/PURL-scoped behavioral contracts, a typed time-decayed graph, and an plain causal-estimation eli-
gibility pathway; the current check doesn't establish that this combination boosts attribution. Anchore Enterprise uses SBOM-centered assessment for vulnerability and supply-chain management [12], while Ox Security describes software supply-chain risk analysis across the development and production contexts [13]. Those capabilities tackle fundamental inventory and risk ques-
tions. PHANTOM focuses on a narrower gap: the package can remain in the SBOM under the same name and version while its runtime behavior changes. Both artifact identity and live behavior are therefore needed by a solid response, with uncertainty retained when an event can't be mapped confidently to a component. This difference matters because package identity is not a behavioral invariant. A PURL normally spots package type, namespace, name, and version; it doesn't prove that a rolled out file is byte-for-byte same to reviewed source or artifact. Stronger provenance can be established by digest pinning and signature verification, but even a validly signed package may include harmful code, as supply-chain incidents have showed. Runtime note adds to those controls by inspecting what loaded software does in the environment where it performs. It can't replace provenance: an event-to-PURL associ-
ation is only as solid as the image metadata, process mapping, and file attribution used to make it. These provenance links must be preserved by a practical framework and expose missing links to the analyst rather than silently allocating an event to the nearest package. The same difference shapes PHANTOM's handling of gentle change. Library upgrades, feature flags, configura-
tion changes, and workload autoscaling can all change event frequencies without showing compromise. A contract trained on one operating envelope may therefore create alerts when a service changes legitimately. Versioned contracts, workload context, and analyst review are needed to separate routine evolution from suspect drift. We treat contract violation as proof needing explanation, not a verdict that a package is malicious. This policy is chiefly important for services whose legitimate behavior is adaptive or customer-specific. D. Causal Inference Applied to Security Cloud incident root-cause systems combine operational signals to help explain service incidents; RCA-Copilot, for example, uses an LLM to project a root-cause category and yield an explanation from incident diagnostics [14]. Earlier host-telemetry research programs also study system-level evidence and provenance [18]. Graph-based anomaly detection similarly uses relationships among items to spot patterns that are difficult to see in separated events. Applying a broad causal-inference library to runtime evidence is an integration; novelty depends on the graph construction, identification checks, mapping quality, and evaluation rather than the use of DoWhy alone. The difference is between gathering a component substitution and inter-
vening on that substitution while identifiedly adjusting set fixed. PHANTOM uses Pearl's intervention notation and DoWhy's causal workflow to represent assumptions behind a component-level estimate [15], [16]. An anomaly score or ranked explanation doesn't by itself spot an intervention effect. SBOM generators establish inventories; vulnerability scanners connect inventories to advisories; runtime mon-
itors collect events and apply policies; incident tools match up operational signals. PHANTOM targets the in-
terfaces among these layers, so it doesn't replace provenance verification, vulnerability management, or runtime enforcement. Eval-
uation should compare contracts keyed only by PURL with models regulated on workload role or service mode, and should compare temporal Markov models with frequency baselines using matched event access. Contract generalization remains an open question. The repo uses PURL identifiers as component keys, following the package URL specification [26]. E. Comparison with Existing Work Table I recaps recorded capability categories for representative systems, and product features rest on deployment and integrations. PHANTOM's entries describe the repository code rather than empirically checked performance; in turn, this table compares stated capabilities, not measured potency. A performance comparison would need happening together runs on same workloads with

the same attack and benign-control labels. The available baseline log records a dry run: Falco and Trivy setup were skipped,

Capabilities reflect recorded design goals; deployment-specific
integrations may extend person tools beyond what's shown, at least in principle. Falco alert output is absent, and the
Isolation Forest
result used synthetic frequency data rather than the saved
attack traces, in actual application, and trivy statically is analyzing
and would need a separately defined, task-appropriate outcome. The current repo artifacts therefore support no claim that PHANTOM outperforms these tools; a representative
workload such as Online Boutique [17]
could be used in a future controlled comparison; it's not part of the six-run dataset. F. Gap This Paper Tackles
The existing system categories are tackling different parts of the problem - runtime event collection and policy enforcement, component inventory and provenance, or incident-level diagnosis; in fact, the current entries
can't answer that question, because they lack PURL-bound
events and component-
level ground truth. The tested
question is whether settled component identity and a defensible temporal causal model can support more accurate,
useful explanations than event presence or graph ranking alone. PHANTOM is exploring an interface
among these categories by representing image/PURL-scoped contracts, runtime relationships, and an explicit causal-query abstention status in one code; this paper doesn't claim that no published system offers any of these capabilities, nor that the combination has been verified to
improve attribution. III. System Design

A. Threat Model

PHANTOM detects runtime drift associated with a software component whose behavior has changed relative to a learned or permitted contract; this includes a malicious replacement that's retaining a familiar package name and version, dependency confusion that introduces an unexpected package into a workload, build-pipeline tampering that's producing behavioral changes, and backdoored libraries that preserve their public API, an application programming interface. The rollout maps workload artifacts and SBOM PURLs to gathered processes; event-level evidence is being enabled by this to be associated with component identities, and b. Behavioral Contract

---

Algorithm 1. Behavioral contract construction

Require: Sequences partitioned by (d,p,[t0,t1]), image digest d, PURL p, token sequences S, minimum Nmin, maximum order Kmax
Ensure: Signed model Cd,p or failure
1: N <- total number of tokens in S
2: if N < Nmin then
3: return failure
4: end if
5: Sigma <- gathered tokens plus reserved unknown/resource tokens
6: for k = 0 to Kmax do
7: Estimate order-k transition counts and BIC score
8: end for
9: k* <- arg min_k BIC(k)
10: for all retained histories h do
11: p_hat(x|h) <- (n(h,x)+1)/(n(h)+|Sigma|)
12: end for
13: Bind model to d,p and provenance metadata; serialize and hash
14: Sign packed model and persist contract record
15: return Cd,p

Generation For a component spotted by a PURL, the contract-
generation use case retrieves training sequences scoped by image digest, PURL and time interval. Candidate orders are checked using BIC with qk = | Hk|(| Σ|− 1) guidelines, and contexts are pruned when their local BIC doesn't improve over a kept suffix; and thus, laplace smoothing is ˆp(x| h) = n(h,x) + 1n(h) +| Σ|, h = 1n(h) +| Σ|. Each context h, the model is estimating next-token probabilities; and thus, the implementation doesn't by itself establish that the training interval contains only normal behavior; that interval and its approval must be controlled by the operator. The Markov model operates over the gathered token alphabet plus reserved unknown/resource tokens; a token is a four-tuple (e) = (t,o,r,p) for event type, operation class, resource class, and privilege class. (1) For each gathered context h, the scorer is computing conditional KL divergence and weights it by the fraction of events gathered in that context, like DKL = h nh N x ˆPobs(x| h) log ˆPobs(x| h)Pcontract(x| h). (2) The code reports novelty separately and applies an optional event-loss multiplier; in fact, the scorer does not apply the split-conformal threshold described in the handoff, so this paper doesn't claim a adjusted binary drift decision or false-alarm guarantee. Contract serialization is hashed, signed through the cosign signer port, and maintained; in fact, the contract model is recording image digest, PURL, model digest, signature bundle, signing identity, selected order,

BIC, training count, and creation time, all things considered. Training
data or
signature integrity doesn't establish baseline
correctness; in check, event identities that can't be settled
must remain unattached to a PURL, to a significant degree. Context
sorting, pruning and parent

TABLE I: Capability comparison against related systems (design-level comparison, not a performance benchmark)

System	eBPF collection	SBOM awareness	Runtime behavior model	Causal attribution	Pre-compromise prediction
Falco	Yes	No	Rules	No	No
Tetragon	Yes	No	Policies/events	No	No
Tracee	Yes	No	Policies/events	No	No
CycloneDX / Anchore	No	Yes	No	No	Vulnerability/risk analysis
Bomfather	Yes (build-time)	Yes	Build connection observation	No	No [29]
eBPF + IMA	Yes	Yes	SBOM enforcement	No	No [30]
KubeArmor	eBPF/BPF	No	Policies	No	No [19]
LLM-based RCA	Varies	No	Incident signals	Ranked explanation	Varies
PHANTOM	Yes	Yes	Component contracts	DoWhy pathway	PCEPS setup

lookup add up to O(Q logQ + QKmax); materializing
smoothed probabilities costs O(H| Σ|); in fact, fine then a conservative
bound is O(NK max +Q logQ +QKmax +H| Σ|) time and
O(Q +H| Σ|) space. No sub-millisecond runtime claim is made, because the available artifacts are encompassing no separated
scoring
benchmark; transition counting costs O(NK max) in the worst case. 1) Complexity, like Let N be the total training-token
count,
Kmax the maximum candidate order, Q the number of separate context-token count entries across the candidate
orders, H the number of kept contexts, and | Σ| the alphabet size; in fact, at inference, forming each context costs up to O(K ∗) for selected order K ∗, followed by anticipated constant-time count lookups and check of gathered context-token pairs. A window of length therefore costs O(K ∗ +Qw) time and O(Qw) short-term space,
where Qw is the number of distinct gathered context-token pairs; c. Behavioral Reliance Graph
We specify the BDG as G = (V,E, ,), where V is a typed entity set, E is a typed multiedge set, maps nodes to gathered attributes. Applied relations include runs, performs, belongs_to, reads, writes, connects_to, derived_from, and violates; in fact, node types include workload, container, process, PURL, file, network endpoint, contract and
drift event. Several edges are kept;
the implementation is using a NetworkX MultiDiGraph representation; what's more, maps edges to event
time and confidence. At each observation time, an edge weight decays
according to

we(t) = (t−tprev)/ we(tprev) +qe, (3)
where 0< < 1 controls forgetting, is the time scale,; as, well, qe is the new evidence contribution. The temporal
projection is designating semantic variables to six applied tiers, namely environment/RBAC/image (0), component version
(1), process behavior (2), file/network/privilege behavior (3), contract deviation (4), and runtime SBOM drift (5). In a person
time window, the code permits an
edge only from a lower to a higher tier; cross-window edges
point from the earlier to later window; this fixed ordering is a
modeling assumption motivated by the guessed supply-chain sequence, not
proof that the ordered variables are causally related in every workload;
it can orient simultaneous notes to generate a DAG, but doesn't by itself establish identification or causal
truth. The BDG is keeping cycles because runtime interactions
can be cyclic; in fact, projection code conservatively is excluding
unsupported orientations and keeps back attribution when
an unresolved cycle entails the treatment or outcome. D. Structural
Causal Attribution
The estimator represents a candidate component change as treatmentS, a binary drift outcome as D, and measured pre-treatment covariates as X. The target is an average treatment effect E[D | do(S = 1)] − E[D | do(S = 0)], under the supplied causal graph and adjustment set. The outcome
follows from the input causal observations;
covariates are supplied by the caller; the BDG projection is expanding
semantic variable classes, across a fixed number of windows. Edges are

TABLE II: Used PCEPS feature schema and source

Slot	Feature	Input source
1	ATE-derived causal effect	Causal attribution result
2	Attribution confidence	Causal attribution result
3	Contract-violation rate	Contract validator
4	Behavioral divergence ratio D/ max(,)	KL drift scorer and contract threshold
5	New-process rate	Process events and contract
6	Unexpected-network rate	Network events and contract
7	Unexpected privilege transition	Privilege events and policy context
8	Sensitive-file violation rate	File events and policy context
9	Component criticality	Component policy input
10	Invalid image signature	SBOM/image verification status
11	Namespace risk weight	Namespace policy input
12	Service-account privilege	Kubernetes RBAC context
13	Event-loss rate	Collector loss counters
14	Graph-centrality delta	BDG snapshot and baseline
15	Prior drift frequency	Prior scored windows
16	Runtime-component novelty	Runtime-to-SBOM identity resolution

added across the strictly increasing time windows, or, within a window
when a hard-coded semantic tier increases; the SCM builder then collapses window labels to semantic variable names, adds treatment-to-outcome and covariate-to-treatment/outcome edges, checks variation and a positivity rule, and serializes the graph for DoWhy. DoWhy identification is requested with unidentifiable effects disallowed [15], [16]; in fact, the setup returns not_identifiable on treatment/outcome cycles, no fluctuation, positivity failure,
construction, identification, or estimation errors. Tests exercise cycle and no-variation abstention paths; these tests are validating software behavior, under the built inputs,
not the correctness of causal assumptions in rollout; and thus, for
completed models, the arranged estimator returns
an ATE and, when available, an interval. The pathway attempts a treated-cohort counterfactual and runs random-common-cause, placebo-treatment, and data-subset refutations; and thus, its confidence value is a heuristic product involving event-loss rate, identity resolution, contract verification, interval width, and refutation pass fraction; it isn't a

adjusted probability of causal correctness, in the main. The graph,
covariates, and unmeasured confounding assumptions govern
interpretation. The run artifacts contain no autonomous component-level ground truth, so attribution accuracy is not reported, at least for now; indeed, e. PCEPS Risk Scorer
PCEPS is a separate analyst-triage pathway, not an attack detector. Table II lists the applied feature schema. Features are clamped to [0, 1]; missing features are represented with a mask and blamed from training medians, except selected conservative defaults that are using
one; in fact, the scorer concatenates 16 values and 16 mask
indicators, gets an XGBoost output p [24], and applies the used Platt mapping [25]:
pcal = (a logit(clamp(p)) +b), PCEPS = 100pcal. (4)
The implementation clamps p to [10−7, 1− 10−7] before the logit; the resulting score lies in [0, 100] and is mapped to fixed severity bands at 20, 40, 60, and 80, and although pcal is a classifier probability output, the available project artifacts don't establish calibration or
future-compromise
prediction. PCEPS is intended to put first review when multiple indicators co-occur; a high score doesn't establish compromise, causal responsibility, or calibrated probability; predictive validity is remaining untested, because no
trained model, held-out labels, or calibration report comes with the run artifacts. IV. Setup
The repo contains SBOM, eBPF, causal-engine, API-gateway, and report-generator services, with a React/TypeScript analyst interface; indeed, the SBOM service integrates Syft and Cosign interfaces for generation and verification [20], [21], [27]. The README documents Python 3.11+, Docker Compose, Terraform/Helm, and Linux CO-RE requirements [22], [28]; the infrastructure tree includes an A WS EKS deployment; and thus, a repo URL, a web
address,
and dataset DOI weren't available in the checked-in materials. The DoWhy adapter delegates to the causal use
case and maps identification and estimation failures to not_identifiable; no correlation fallback is used as a causal
estimate; the API gateway exposes operations to the analyst
client, and the report generator puts together evidence for
later review. Event schemas are versioned across the service
boundaries; indeed, the graph implementation uses NetworkX's directed
multigraph representation [23]. The D3 graph view exposes graph nodes, edges and evidence records for analyst inspection, and its visualization doesn't add
autonomous

evidence for a causal claim. V. Check

A. Experimental Setup

The checked-in raw3 analysis set contains nine JSON entries dated September 5, 2026; six data contain a completed attack phase, namely three ‘xzutils-001‘ runs and three ‘dep-confusion-001‘ runs. Three SolarWinds-style ‘solarwinds-001‘ entries report ‘inject() returned False‘ and have no attack phase; indeed, they're failed trials, not de-
tection outcomes. The successful entries include injection and first-event timestamps, raw drift-event data, and phase metadata, and the recorded detection window is 120 s after injection, as specified in the raw scenario records and evaluation code. The older ‘raw/‘ directory contains 114 legacy JSON records, including entries labelled for harmless cases, but the data inspected phantomly detect and include ‘dry-run-pod‘ placeholders; they can't serve as live benign controls; in fact, although the run script is describing a planned EKS setup, the raw3 files don't provide a complete cluster/kernel/container/image-digest clear. The saved event records spot a cluster name but have vague or missing pod identity and no SBOM binding; indeed, b. RQ1: Detection Performance All six completed attack runs are marked event-positive, because at least one drift event was recorded, within the detection window; this gives an event-positive frac- tion of 6/6 (1.000; Wilson 95% interval [0.610, 1.000]), and it doesn't establish component-level attack detection, like the recorded events are namespace-change or privilege- transition events, and their runtime identity is unclear or missing, with no PURL or SBOM ID. The three failed SolarWinds-style injections don't count as false negatives, and the evidence therefore supports only event-
presence counts for the two completed families. Latency is measured from injection timestamp to first recorded PHANTOM event; the recomputed mean is 15.737 s, median 5.516 s, and linearly interpolated P95

Note: Completed attack runs unresolvedly identified; all three build-pipeline injections failed; no benign controls were
recorded. 54.718 s, rounding to 15.74, 5.52. What's more, the saved experiment set includes no
trained PCEPS artifacts. More than the rest,
the drift scorer has no implemented alert threshold, causal eligibility is conditional on the supplied data and graph assumptions. The listing connects implemented service pathways; it
doesn't suggest an end-to-end evaluation. The event rows are copied in the saved API
output, so the unit is the scenario run, not 3,701 event rows. As, the dataset contains no benign controls, FPR, specificity, precision, F1, and false positives per hour are undefined; these values are recorded as unavailable rather than set to

TABLE III: Experimental configuration and run manifest

Parameter	Evidence in saved artifacts
EC2 instance type	Not recorded
Linux kernel	Not recorded in raw3 manifest
Kubernetes version	Not recorded in raw3 manifest
Container runtime/version	Not recorded
Experiment commit	Not linked to raw3 records
Python / DoWhy versions	Repository requires Python 3.11+; experiment versions aren't recorded
Completed attacks	xzutils-001: 3; dep-confusion-001: 3
Failed injections	solarwinds-001: 3 ('inject() returned False')
Detection window	120 s post-injection
Baseline outputs	Dry-run record only; no comparable raw detections
Benign controls	None in current dataset
PURL/SBOM binding	Unresolved in saved event records
CPU/memory overhead	Not measured; saved snapshots are zero without provenance

TABLE IV: Run-level outcomes by attack family

Attack family	Valid	Event+	Failed
XZ-style backdoor	3	3	0
Dependency confusion	3	3	0
Build-pipeline tampering	0	0	3
Benign controls	0	N/A	N/A

zero. The first XZ run has a
70.332 s latency; available entries don't establish its
cause. And 54.72 s. Each stored
MTTD is matching its timestamps. The six-run Wilson interval is
extensive;
broader evaluation is needed before generalizing beyond the tested runs. C. RQ2: Causal Attribution Accuracy The checked-in aggregate reports attribution accuracy of 1.0. Though, the fundamental records don't contain per-run
anticipated PURLs or a comparison with the oracle target PURL. All saved drift events ambiguously identify, null PURL/SBOM
fields, and unknown workload
identifiers. Component identity and causal correctness therefore can't be verified. And no attribution accuracy
is reported.

D. RQ3: PCEPS Prediction

The PCEPS source is implementing feature extraction, XG-
Boost training, Platt scaling, Brier/ECE calculations, and scoring. No trained PCEPS model, calibration artifact, scored windows, labels, or metric output supporting the manuscript's former AUC/Brier values was found. No predictive-performance result is mainly reported. The score's future-outcome interpretation requires a separate prospec-

Algorithm 2. Evidence-gated PHANTOM processing pathway

Require: SBOM and image identity evidence; normalized event window W_i; available contract, BDG, causal, and PCEPS artifacts
Ensure: Evidence report with clear unavailable or unsettled fields
1: Verify SBOM digest and signature; reject invalid artifacts
2: Normalize supported events and resolve workload/process identity
3: if component identity is supported by verified evidence then
4: associate events with image digest and PURL
5: else
6: keep unresolved identity; don't designate a PURL
7: end if
8: if a valid contract is available then
9: compute the implemented KL score and novelty indicators for W
10: else
11: mark contract comparison unavailable
12: end if
13: update the typed BDG, ignoring duplicate event UUIDs and using edge decay
14: if causal observations satisfy implemented graph and data eligibility checks then
15: estimate the configured DoWhy effect and run available refutations
16: else
17: return not_identifiable with the diagnostic reason
18: end if
19: if trained PCEPS model and calibration parameters are available then
20: find the bounded score from the 16-characteristic vector and missingness mask
21: else
22: mark PCEPS unavailable
23: end if
24: create a report from recorded evidence and unre-tackled fields

tive evaluation. E. RQ4: Overhead The saved CPU, the main processor and memory, alongside event-lag snapshots are all zero. No raw Prometheus samples or measurement

provenance are accompanying them; these entries cannot es- tablish zero resource cost or a particular event-loss rate; runtime overhead is remaining unmeasured, at least in practice; indeed, f. Suggested Benign-Control Protocol Benign controls are needed to estimate how often ordinary workload changes trigger the event or contract pipeline, to a significant degree. False-positive rate, precision, specificity, and F1 are undefined; an event-positive fraction of 6/6 is not a recall estimate, because its event oracle does not need a correctly credited attack, until these runs are collected; these three scenario classes are a suggested protocol, not checked existing runner functionality. A future study should encompass routine package modifies, load changes without component changes, and pod restart/autoscaling episodes; these controls should use the same observation duration and instrumentation as attack runs, with labels fixed before examination; the project repo and saved experiment set do not establish that benign-update, benign-load, and benign-restart automation has been applied or carried out. A future check should pre-register the workload, contract version, event labels, and alert threshold, then repeat each class under the matched workload and rollout conditions; sample size should be picked from the desired confidence interval rather than treating three runs per class as automatically sufficient. The current data support only that drift events were recorded during six completed attack windows; and thus, they do not bound harmless alert rates or establish attack detection exactness. The three failed SolarWinds-style injections must be fixed and rerun before that attack family can, as it stands, be assessed, and g. Threats to Validity The six completed runs cover two attack families and use a run-level event-presence label; event presence does not establish that a malicious event was, put simply, credited to the injected component. The baseline log documents a dry run, including Falco and Trivy setup were skipped and Falco alert logs were missing. Three attack injections failed. No benign controls, all-zero extra costs snapshots, unresolved component identity, and no reproducibility clear with kernel/runtime versions and image digests are had by The raw data; trivy is a static vulnerability scanner, so a runtime detection comparison needs a separately defined task and labeling rule. PCEPS model/calibration outputs and causal ground-truth comparisons are absent, so these gaps prevent claims of comparative accuracy, causal attribution quality, calibration and operational running costs; and thus, the causal projection also hinges on caller-supplied covariates, hard-coded semantic tiers, and a DAG that includes treatment-to-outcome and covariate edges by construction. The resulting estimate is conditional on those assumptions; neither temporal precedence nor a graph path is proving causation; the gathered event latency cannot be generalized to rollout performance without matched controls, repeated workloads, and recorded environment details. The attack sample is small, run-level labels measure event presence rather than correctly credited compromise, and failed injections are left out, and existing tests exercise specified abstention paths, but do not test causal correctness against intervention ground truth. VI. Conclusion PHANTOM is showing an applied pathway that is connecting SBOM-facing contract generation, eBPF event collection, typed graph maintenance, and a causal-estimation interface with plain abstention; the immediate evaluation priorities are gentle controls, per-run component-ground-truth entries, repeatable SolarWinds-style injections, matched baseline output, and a trained PCEPS model found on held-out labels. A local run of the core and five service test suites completed with 454 tests passed and one skipped; this checks exercised software behaviors, not the validity of component-level causal claims in a live rollout; indeed, event-positive outcomes in six completed runs are shown by The experiment records, but do not prove package mapping, attribution accuracy, possible risk prediction, gentle false-positive behaviour, or overhead. Kernel, Kubernetes, container runtime, tool versions, image digests, and experiment commit must be recorded with the rerun; indeed, no claim is made that the infrastructure for all these experiments is already applied or ready to carry out. Supply-chain incidents expose a gap between declared software inventory and gathered runtime activity; and thus, PHAN-

TOM looks into one way to represent that gap while making missing identity and failed causal identification visible to an analyst. Its present insight is an used research prototype and an auditable account of what the available tests and six successful attack runs do, and do not, show; in fact, controlled check must establish whether its component-scoped contracts and causal pathway produce solid attribution beyond event detection.

References

[1] CISA, "Reported Supply Chain Compromise Affecting XZ Utils Data Compression Library, CVE-2024-3094," Alert AA24-089A, Mar. 29, 2024.

[2] A. Freund, "Backdoor in upstream xz/liblzma leading to ssh server compromise," oss-security mailing list, Mar. 29, 2024. [Online]. A vailable: https://www.openwall.com/lists/ oss-security/2024/03/29/4

[3] CISA, "Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations," Alert AA20-352A, Dec. 2020. [Online]. A vailable: https://www.cisa.gov/news-events/ cybersecurity-advisories/aa20-352a

[4] FireEye, "Highly Evasive Attacker Leverages SolarWinds Sup- ply Chain to Compromise Multiple Global Victims With SUNBURST Backdoor," Dec. 2020.

[5] Apache Software Foundation, "Apache Log4j Security Vul- nerabilities: CVE-2021-44228," Dec. 2021. [Online]. A vailable: https://logging.apache.org/log4j/2.x/security.html

[6] National Vulnerability Database, "CVE-2021-44228 Detail," NIST, 2021. [Online]. A vailable: https://nvd.nist.gov/vuln/ detail/CVE-2021-44228

[7] Sysdig, "Falco: Cloud Native Runtime Security," CNCF project documentation. [Online]. A vailable: https://falco.org/ docs/ (Accessed: Oct. 16, 2026).

[8] Isovalent, "Tetragon: eBPF-Based Security Observability and Runtime Enforcement," 2023. [Online]. A vailable: https:// tetragon.io/ (Accessed: Oct. 16, 2026).

[9] Aqua Security, "Tracee: Linux Runtime Security and Forensics Using eBPF," 2022. [Online]. A vailable: https://github.com/ aquasecurity/tracee (Accessed: Oct. 16, 2026).

[10] OW ASP Foundation, "CycloneDX Specification, Version 1.4," 2022. [Online]. A vailable: https://cyclonedx.org/specification/ archive/ (Accessed: Oct. 16, 2026).

[11] U.S. Department of Commerce, "The Minimum Elements for a Software Bill of Materials (SBOM)," NTIA, 2021. [Online]. A vailable: https://www.ntia.gov/report/2021/ minimum-elements-software-bill-materials-sbom

[12] Anchore Inc. "Anchore Enterprise: SBOM-Powered Software Supply Chain Security," 2023. [Online]. A vailable: https:// anchore.com/enterprise/ (Accessed: Oct. 16, 2026).

[13] Ox Security, "Software Supply Chain Security Platform," product documentation. [Online]. A vailable: https://ox.security/ (Accessed: Oct. 16, 2026).

[14] Y. Chen et al. "Automatic Root Cause Analysis via Large Language Models for Cloud Incidents," in Proc. 19th European Conf. Computer Systems (EuroSys), Athens, Greece, Apr. 2024, pp. 674-688.

[15] J. Pearl, Causality: Models, Reasoning, and Inference, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2009.

[16] A. Sharma and E. Kiciman, "DoWhy: An End-to-End Library for Causal Inference," arXiv:2011.04216, 2020. [Online]. A vailable: https://arxiv.org/abs/2011.04216

[17] Google LLC, "Online Boutique: Cloud-Native Microser- vices Demo," GitHub repository. [Online]. A vailable: https://github.com/GoogleCloudPlatform/microservices-demo (Accessed: Oct. 16, 2026).

[18] DARPA, "Transparent Computing Program: Data and Resources," program materials. [Online]. A vailable: https://www.darpa.mil/research/programs/transparent-computing (Accessed: Oct. 16, 2026).

[19] AccuKnox, "KubeArmor: Container-Aware AppArmor, SELinux, and BPF Security," 2022. [Online]. A vailable: https://kubearmor.io/ (Accessed: Oct. 16, 2026).

[20] Anchore, "Syft: Generate a Software Bill of Materials from Container Images and Filesystems," project documentation. [Online]. A vailable: https://github.com/anchore/syft (Accessed: Oct. 16, 2026).

[21] Sigstore, "Cosign: Container Signing, Verification, and Storage," project documentation. [Online]. A vailable: https://docs. sigstore.dev/cosign/ (Accessed: Oct. 16, 2026).

[22] eBPF Foundation, "What is eBPF?" project documentation. [Online]. A vailable: https://ebpf.io/what-is-ebpf/ (Accessed: Oct. 16, 2026).

[23] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in Proc. SciPy, 2008, pp. 11-15.

[24] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD, 2016, pp. 785-794.

[25] J. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in Advances in Large Margin Classifiers, 1999, pp. 61-74.

[26] Package-URL Working Group, “Package URL (PURL) Spec- ification,” project documentation. [Online]. A vailable: https: //github.com/package-url/purl-spec (Accessed: Oct. 16, 2026).

[27] Sigstore, “Cosign Documentation: Signing Containers,” project documentation. [Online]. A vailable: https://docs.sigstore.dev/cosign/signing/signing_containers/ (Accessed: Oct. 16, 2026).

[28] Linux Foundation, “libbpf: BPF CO-RE Reference Guide,” project documentation. [Online]. A vailable: https://github. com/libbpf/libbpf (Accessed: Oct. 16, 2026).

[29] N. Srinivasan, N. Naveen, and N. Naveen, “Bomfather: An eBPF-based Kernel-level Monitoring Framework for Accu-
rate Identification of Unknown, Unused, and Dynamically Loaded Dependencies in Modern Software Supply Chains,” arXiv:2503.02097, 2025. [Online]. A vailable: https://arxiv.org/ abs/2503.02097.

[30] A. Crawford, E. Yakubovich, and R. Szumski, “Enforcing SBOMs through the Linux kernel with eBPF and IMA,” in Proc. ACM Workshop on Software Supply Chain Security (SCORED), 2023, pp. 77-78, doi: 10.1145/3605770.3625206.